Articles

overview

FAQ
const(FAQ)
Floating Point
Warnings
Rationale
Builtin Rationale
C to D
C++ to D
C Preprocessor vs D
Code coverage analysis
Exception Safety
Hijacking
Introduction to std.datetime
Lazy Evaluation
Migrating to Shared
String Mixins
Regular Expressions
SafeD
Templates Revisited
Template Constraints
Compile-time Sequences
Variadic Templates
Template Comparison
D Slices
D's Contract Programming
Writing Shared Libraries for Linux
Coralling Wild Pointers With ref return scope
Creating Windows DLLs

String Mixins

String Mixins (not to be confused with template mixins) enable string constants to be compiled as regular D code and inserted into the program. Combining this with compile time manipulation of strings enables the creation of domain-specific languages.

For example, here we can create a template that generates a struct with the named members:

```
template GenStruct(string Name, string M1)
{
    const char[] GenStruct = "struct " ~ Name ~ "{ int "
}

mixin(GenStruct!("Foo", "bar"));
```

which generates:

```
struct Foo { int bar; }
```

Superficially, since D mixins can manipulate text and compile the result, it has some similar properties to the C preprocessor. But there are major, fundamental differences:

- The C preprocessing step occurs **before** lexical analysis. This makes it impossible to lex or parse C without access to all of the context, including all #include'd files, paths and all relevant compiler switches. Mixins occur during semantic analysis, and do not affect the lexing or parsing process. Lexing and parsing can still occur without semantic analysis.
- The C preprocessor can be used to create what appears to be different syntax:

  ```
  #define BEGIN {
  #define END   }

  BEGIN
    int x = 3;
    foo(x);
  END
  ```

  This monkey business is impossible with mixins. Mixed in text must form complete declarations, statements, or expressions.
- C macros will affect everything following that has the same name, even if they are in nested scopes. C macros cut across all scopes. This problem is called being not "coding hygenic". Mixins follow the usual scoping rules, and is hygenic.
- C preprocessing expressions follow a different syntax and have different semantic rules than the C language. The C preprocessor is technically a different language. Mixins are in the same language.
- C const declarations and C++ templates are invisible to C preprocessing. Mixins can be manipulated using templates and const declarations.